Intrusion Detection Systems (IDS): A study of machine learning integration

Network Security Report

Diogo Salazar Ramos
Tiago Miguel Alves Pires

Master in Network and Information Systems Engineering

December 8, 2025

Contents

1 Introduction

Following the conclusion of a comprehensive theoretical analysis concerning Intrusion Detection Systems (IDS) and their underlying methodologies, the present report shifts focus to the practical dimension: the **construction and validation of a functional IDS** utilizing Machine Learning (ML) techniques. The primary objective is to execute a Proof-of-Concept (PoC) to demonstrate the real-time efficacy of a classification model in identifying malicious traffic within a simulated network environment.

The adoption of Machine Learning in intrusion detection systems is justified by the necessity to overcome the limitations inherent in traditional methods:

- **Scalability and Complexity:** Machine Learning enables the efficient analysis of a vast number of traffic *flow features* at high speeds. This capability is essential for managing the increasing complexity and the massive data volume characteristic of modern networks.
- **Dynamic Adaptability:** Models such as the **Decision Tree (DT)** and the **Random Forest (RF)** have proven to be highly effective in the rapid discrimination between benign and attack traffic patterns, as demonstrated in the preceding evaluation phase.

2 Dataset

2.1 Dataset Selection

The dataset used in this work was the CIC-IDS2017, specifically the subset *Wednesday-workingHours*. This dataset is widely recognized in the Intrusion Detection System research environment because it contains realistic network traffic, including both benign network traffic and a wide range of attack types. The usage of a subset was decided based on:

- **Reducing computational complexity**, due to the large size of CIC-IDS2017 dataset, working with a subset allows for a more efficient model training, testing and comparison while keeping a healthy variety of possible attacks

This subset contains benign network traffic and several types of Denial of Service (DoS) attacks, including:

- **DoS Hulk**
- **DoS GoldenEye**

- **DoS Slowloris**
- **DoS Slowhttptest**
- **Heartbleed**

A relevant challenge when working with this dataset is the clear class imbalance. While Benign has 400,000 entries, some attacks like Heartbleed only have 11 samples. This imbalance can negatively affect the performance of the trained model on minority classes if proper Data Preprocessing techniques aren't leveraged.

2.2 Data Preprocessing

Encoding and Normalization

Data Preprocessing was a crucial step before training the models to ensure a optimal model performance. Although most features in the subset are numerical, the target variable "Label" is categorical. To address this we encoded those entries into integer representations to be compatible with the machine learning models used. In addition, all numerical features were normalized to the range [0,1]:

- preventing large ranges features to dominate smaller range features,
- improve performance and stability of the models.

Train-Test Split and SMOTE oversampling

After preprocessing the subset was split into training (80%) and testing (20%) sets using stratification to preserve the original distribution in both sets. To address the class imbalance we had 3 possible solutions:

- **Undersampling** where we remove entries from the most dominant classes, in our case Benign and possibly also DoS Hulk.
- **Oversampling** where we extrapolate new samples for minor classes, in our case Heartbleed.
- Neither, in this case we wouldn't try to balance the dataset but we would weight the target classes, avoiding misclassifying minor classes.

In our project we utilized Undersampling for the dominant class "Benign", reducing the total number of entries from 440,000 to 200,000 in junction with SMOTE (Synthetic Minority Oversampling Technique), an algorithm that was applied to the training set, generating synthetic samples for minor classes:

- reducing bias towards dominant classes

- improving detection performance for rare attacks
- avoiding overfitting by simply duplicating minor class samples

3 Machine Learning Models

3.1 Decision Tree

3.1.1 Overview

A Decision Tree is a classification algorithm used in data mining. Classification is a process of inductively learning a model from a pre-classified dataset, essentially mapping a set of attributes to a particular class.

Decision trees are effective for intrusion detection because they treat it as a classification problem, learning a model from data to classify network activity as normal or malicious. They are suitable for real-time use due to their high performance with large datasets, offer easily interpretable models for security officers, and possess strong generalization accuracy to detect new, variant attacks. [1]

3.1.2 Performance and Results

1. Metrics

Metric	**Value**
Accuracy	0.99941534
Precision	0.99941964
Recall (Sensitivity)	0.99941534
F1-score	0.99941669

Table 1: Decision Tree Model Performance Metrics

2. Analysis

   - **Benign Traffic Assessment (Class 0):** The model achieved 87,955 True Negatives (correctly identified Benign traffic). Crucially, only **51 instances** of benign traffic were incorrectly classified as an attack (False Positives - FP). This low FP rate indicates very high Precision for the Benign class, suggesting the model would generate minimal false alarms.
   - **Attack Detection Capability:** The model produced a total of **12 False Negatives (FN)** across all intrusion classes. This exceptionally low FN count confirms the model's high Recall (Sensitivity).

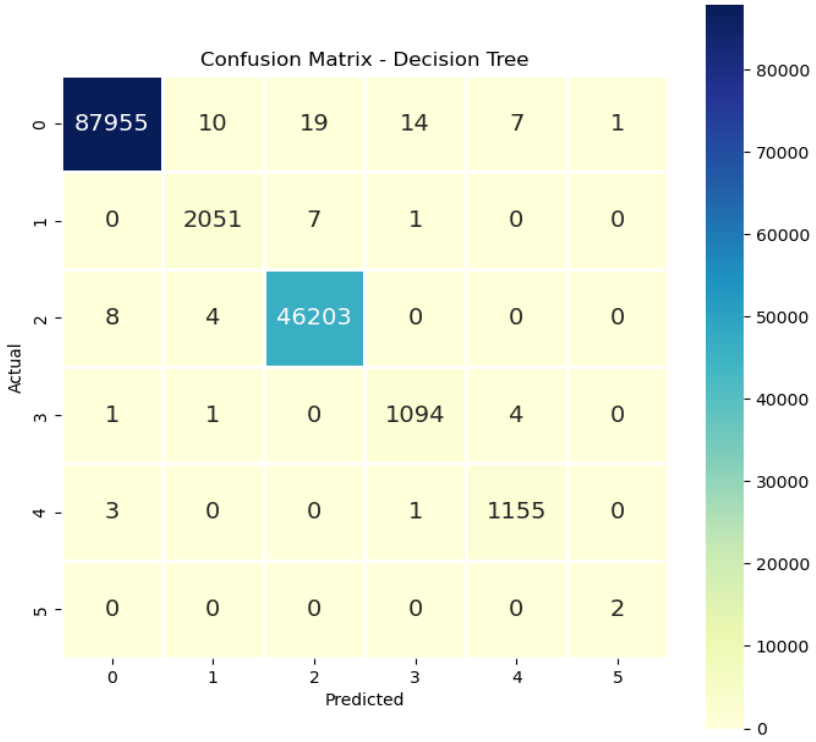

Figure 1: Confusion Matrix - Decision Tree

3.2 Random Forest

3.2.1 Overview

Random Forest is an ensemble supervised machine learning method used for both classification and regression. It improves model performance by growing multiple decision trees rather than just one. Each tree is trained on a random subset of the data. The final prediction is determined by combining the results from all trees, typically through a majority vote for classification. Increasing the number of trees boosts accuracy and reduces overfitting. [2]

3.2.2 Performance and Results

1. Metrics

Metric	Value
Accuracy	0.99937925
Precision	0.99937958
Recall	0.99937925
F1-score	0.99937927

Table 2: Random Forest Model Performance Metrics

2. Analysis

- **Benign Traffic Assessment (Class 0):** The RF generated **57 False Positives**, a marginal increase compared to the DT. The largest source of FP was misclassification into Class 2 (50 instances).
- **Attack Detection Capability:** The RF model produced a total of only **11 False Negatives** across all intrusion classes. This is the **lowest FN count** among all models, demonstrating the highest effective security performance, as minimizing missed attacks is the paramount objective for an IDS.

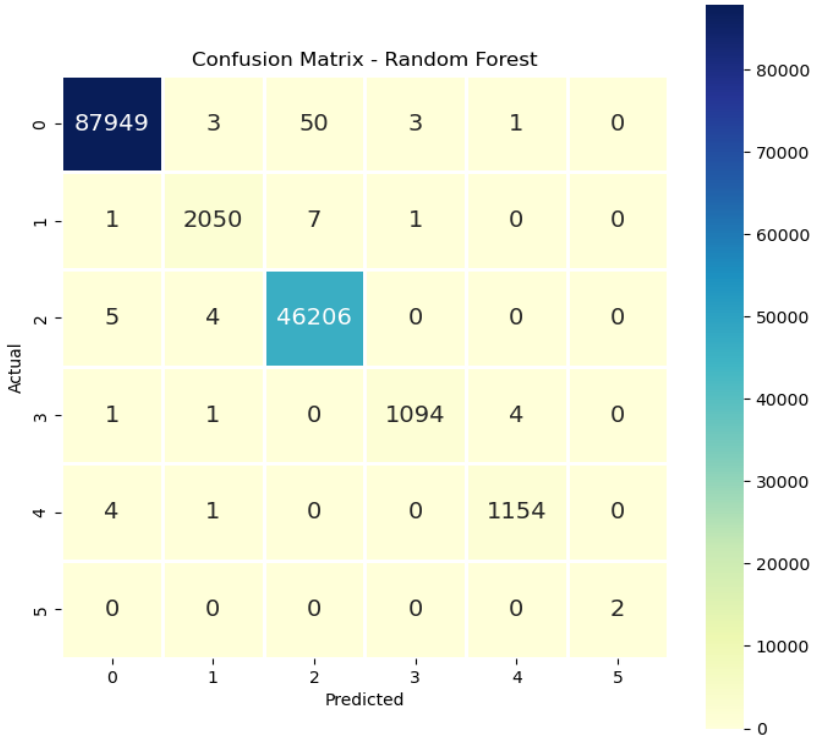

Figure 2: Confusion Matrix - Random Forest

3.3 XGBoost

3.3.1 Overview

Applying XGBoost , a boosting-type ensemble learning method, as a detector within an Intrusion Detection System is a research opportunity, particularly relevant for addressing imbalanced data.

Boosting utilizes multiple weak learners where, in each subsequent iteration, the algorithm iteratively adjusts to previously misclassified data. This sequential method reduces bias and enhances the overall performance of the predictive model.[3]

3.3.2 Performance and Results

1. Metrics

Metric	**Value**
Accuracy	0.99837593
Precision	0.99837878
Recall (Sensitivity)	0.99837593
F1-score	0.99837407

Table 3: XGBoost Model Performance Metrics

2. Analysis

   - **Benign Traffic Assessment (Class 0):** XGBoost produced **153 False Positives**, a rate approximately three times higher than the other models. This high FP rate severely impacts the usability of the IDS due to alert fatigue.
   - **Attack Detection Capability:** The model failed to detect a total of **53 actual attacks** (FN). This count is approximately five times higher than the Random Forest model, indicating a critical lack of Sensitivity (Recall).
   - **Critical Error Location:** The most pronounced failure was in **Class 4**, where 33 actual attacks were incorrectly classified as Benign (Class 0).

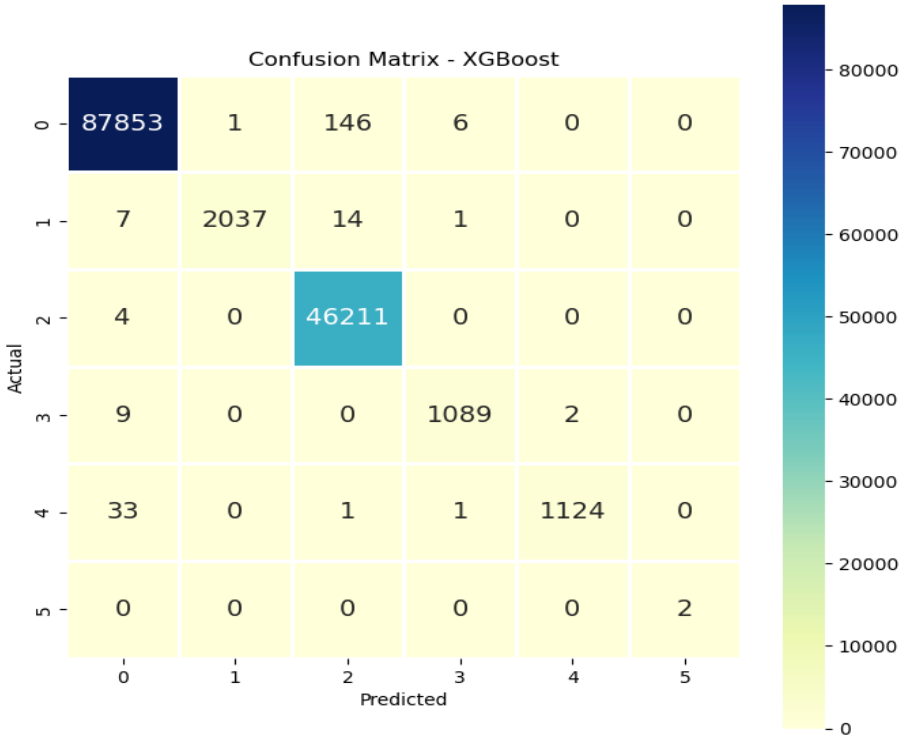

Figure 3: Confusion Matrix - XGBoost

3.4 Comparative Analysis Summary

The comparative analysis reveals a significant performance cluster among the tree-based models, clearly distinguishing them from the boosting ensemble:

- **Performance Similarity (DT and RF):** Both the Decision Tree and the Random Forest models exhibited nearly identical and exceptionally high overall performance metrics. The global Accuracy, Precision, Recall, and F1-score for both models clustered above 0.9993.
- **Security Efficacy Nuances:** While the metrics were similar, the detailed Confusion Matrix analysis highlighted subtle but critical differences. The **Random Forest** model achieved the best security metric by recording the **lowest number of False Negatives (FN=11)**, indicating the highest capacity to correctly identify actual attacks (highest Recall). Conversely, the **Decision Tree** recorded the **lowest number of False Positives (FP=51)**, suggesting a marginally lower false alarm rate.
- **XGBoost Underperformance:** The **XGBoost** model demonstrated

a markedly weaker performance, falling approximately 0.1% below the other two models in global metrics. Critically, it produced the highest number of errors in both security-critical categories: **53** False Negatives and **153** False Positives. This significant failure to minimize both missed attacks and false alarms renders XGBoost unsuitable for this specific IDS application.

3.5 Conclusions

The Random Forest model presents the marginally superior security profile due to its lower FN count and higher inherent robustness against overfitting. However, the requirement for real-time operation in a functional Intrusion Detection System necessitates the consideration of **computational cost** and **inference latency**.

- **Computational Overhead:** Random Forest is an ensemble method that relies on the aggregation of predictions from a large number of individual Decision Trees ($n\_estimators$). This parallel processing inherently imposes a **significantly higher computational load** during the inference phase compared to the execution of a single Decision Tree.
- **Model Choice:** Given the negligible difference in security performance between the two leading models (DT vs. RF), the **Decision Tree** is selected for the practical Proof-of-Concept (PoC). The DT offers the dual advantage of near-perfect accuracy with a **substantially reduced computational footprint**, enabling faster classification throughput essential for time-critical network monitoring.

The subsequent phase of this research will focus on the operational deployment and validation of the Decision Tree model.

4 Proof of Concept

4.1 Introduction and System Architecture

The objective of this Proof of Concept was to establish a functional, end-to-end Machine Learning Intrusion Detection System capable of capturing network traffic, extracting statistical flow features, and classifying network activity as either benign or malicious using a pre-trained Decision Tree model.

The system is deployed across two environments: a Host Machine (Windows) for traffic simulation and a Virtual Machine (VM Linux/Kali) hosting the core IDS pipeline.

4.1.1 The Three-Stage Detection Pipeline

The IDS operates sequentially, with each stage representing a critical transformation of the network data:

1. Capture Stage (VM): Raw network traffic is intercepted from a designated interface.
2. Feature Engineering Stage (VM): The raw traffic is converted into high-dimensional, statistical flow vectors (features) suitable for machine learning.
3. Classification Stage (VM): The engineered features are normalized and fed into the trained Decision Tree model for real-time inference.

4.2 Tooling and Environment

1. **VM Linux**
   - **Tshark -** Command-line utility used to listen on the Host-Only network interface, capturing raw packet data and saving it to a temporary .pcap file
   - **CICFlowMeter -** A Java-based tool that processes the .pcap file. It statistical flow features that serve as the input vector for the ML model
   - **Python Ecosystem -** Utilizes pandas for data manipulation, scikit-learn for the model, and joblib for loading serialized model artifacts
2. **Host Windows**
   - **Scapy -** Python library used to read a pre-recorded attack PCAP from the CIC-IDS dataset and inject those packets into the Host-Only network adapter, simulating a network attack targeting the VM
   - **Npcap -** Npcap provides the necessary raw socket access, enabling Scapy to bypass the Windows kernel and send customized packets over the network interface

4.3 Automation Scripts

- **Host Sender Script -** This is a Python script utilizing Scapy, running on the Host machine. Its function is to initiate the test sequence by:
  1. Establishing a connection to the correct Host-Only Network Adapter.
  2. Replaying the packets from a selected PCAP file, directing them to the known interface of the VM.

- **VM IDS Script -** This Bash script manages the complete IDS lifecycle on the VM, ensuring correct command execution and sequential data handling:

    1. Executes Tshark with sudo permissions to capture traffic for a specified interface
    2. Calls the CICFlowMeter binary, feeding it the temporary .pcap file and output a CSV
    3. Invokes the classification script using the explicit Python binary path of the virtual environment. This ensures the correct loading of dependencies and artifacts (model, feature order, and min-max values).

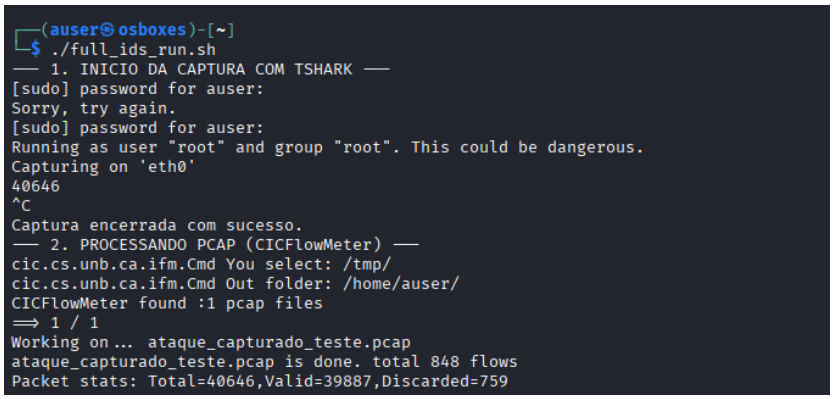

Figure 4: VM Script Execution 1

```
── 3. CLASSIFICAÇÃO DOS FLOWS (IDS Python) ──
/home/auser/ids_venv/lib/python3.13/site-packages/sklearn/base.py:442: Incons
istentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier fro
m version 1.5.1 when using version 1.7.2. This might lead to breaking code or
 invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainabili
ty-limitations
  warnings.warn(

[*] A carregar e pré-processar dados de /home/auser/ataque_capturado_teste.pc
ap_Flow.csv ...
⟶ Inserir 1 feature(s) em falta com valor 0.

[*] Previsões ...

── RELATÓRIO DE DETEÇÃO DE INTRUSÃO (Decision Tree) ──
Predicted_Label
Benign            495
Dos Hulk          247
DoS GoldenEye      58
DoS slowloris      47
Name: count, dtype: int64

[!!!] ALERTA: ATAQUE(S) DETETADO(S)! [!!!]

Primeiros 5 Flows de Ataque:
                                  Flow ID Predicted_Label
0    192.168.10.14-209.48.71.168-49459-80-6        Dos Hulk
2     192.168.10.3-192.168.10.17-88-46124-6   DoS GoldenEye
4     192.168.10.3-192.168.10.17-88-46126-6        Dos Hulk
6     192.168.10.3-192.168.10.17-88-46131-6        Dos Hulk
8  192.168.10.3-192.168.10.15-49666-49413-6        Dos Hulk
```

Figure 5: VM Script Execution 2

```
PS C:\Users\pc\Downloads> python3 ids.py
[*] A iniciar leitura do ficheiro PCAP grande (Limite: 50000 pacotes)
[*] A enviar pacotes via streaming na interface: Ethernet 2...
[*] Pacotes enviados: 50000
[+] Replay concluído.
[+] Total de pacotes enviados: 50000
[+] Duração: 80.53 segundos
```

Figure 6: Host Script Execution

4.4 Challenges and Outcomes

1. **Validation -** The primary obstacle encountered was the inability to directly compare the obtained classification output with the original dataset during the live simulation. Due to time constraints, a system could not be developed to precisely log and map the specific flows sent from the Host with their expected classification results in the VM.

    - **Model Refinement:** It prevented the accurate assessment of the model's performance metrics (True Positive Rate, False Positive Rate)
    - **Script and Artifact Validation:** The inability to definitively validate the output against known-good results limited the refinement

of the script and the overall model.

2. **Execution Flow -** The work successfully established a functional end-to-end execution flow. The project validated the seamless integration of disparate tools—Scapy, Tshark, CICFlowMeter, and Python ML artifacts—into a single, automated pipeline capable of capturing, processing, and classifying network traffic between two distinct operating systems. This successful development of the execution flow conclusively demonstrated the viability of the MLIDS concept.

4.5 Future Work

To transition this PoC into a reliable MLIDS solution, the following areas of improvement are prioritized:

1. **Refinement and Robustness**
   - **Real Dataset Mapping Implementation:** Develop a mechanism to compare the real dataset with the dataset classified in the VM.
   - **Script Evolution:** Based on the verifiable results obtained from the real dataset mapping, the classification script can be evolved. This comparison will allow for the systematic identification and correction of errors introduced during the capture, processing, or normalization stages, leading to a much more accurate system.
   - **Model Refinement:** Following the successful establishment of the validation framework, the Decision Tree model can be significantly improved and optimized.
2. **Increased Automation and Stability:**
   - **Continuous Monitoring:** Enhance the Bash script to implement a continuous loop of capture-process-classify cycles, allowing the system to operate as a true, near-real-time IDS.
   - **Robust Pre-processing:** Implement more sophisticated logging and error handling in to better manage NaN and Infinity values from the CICFlowMeter, ensuring stability in noisy network environments.

5 Conclusion

This work examined the use of Machine Learning within IDS, evaluating Decision Tree, Random Forest and XGBoost models under a unified preprocessing pipeline. Ultimately the Decision Tree was chosen for implementation due to its strong metrics and low computational cost.

A Proof of Concept was implemented using a two machine setup in which one injected .pcap traffic while the other captured, processed it using CICFlowMeter and calssified it with the tranined model.

Despite the existing operational challenges and the need for improvements, the project successfully established a functional execution flow. This accomplishment validates the core viability of integrating specialized tools (Tshark, CICFlowMeter, Scapy) and a Machine Learning model into a single, automated pipeline. Given the time constraints, this foundation represents a strong starting point for future work focusing on rigorous model optimization and full system automation.

References

[1] Manish Kumar, M. Hanumanthappa, and T. V. Suresh Kumar. Intrusion detection system using decision tree algorithm. In *2012 IEEE 14th International Conference on Communication Technology*, pages 629–634, 2012.

[2] Emmanuel. C. Uwazie, Afolayan Ayodele Obiniyi, Morufu Olalere, and Perpetua N. Achi. Comparison of random forest, k-nearest neighbor, and support vector machine classifiers for intrusion detection system. *2024 International Conference on Science, Engineering and Business for Driving Sustainable Development Goals (SEB4SDG)*, pages 1–6, 2024.

[3] Aji Gautama Putrada, Nur Alamsyah, Syafrial Fachri Pane, and Mohamad Nurkamal Fauzan. Xgboost for ids on wsn cyber attacks with imbalanced data. *2022 International Symposium on Electronics and Smart Devices (ISESD)*, pages 1–7, 2022.